



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SECURE DIGITAL BANKING & TRANSACTION PROCESSING PLATFORM

PRESENTED BY
JASMITHA R(192524295)

ABSTRACT

Developed a Secure Digital Banking & Transaction Processing Platform for safe and efficient transactions. Enables users to check balance, transfer money, and view history. Built using Python with functions, loops, conditionals, and exception handling. Uses dictionary mapping, JSON/file handling, and string processing for data management and execution. Ensures security, reliability, and easy user interaction.

OBJECTIVE

- To implement the system using Python concepts like functions, loops, and exception handling
- To ensure data security and efficient data management using file handling/JSON

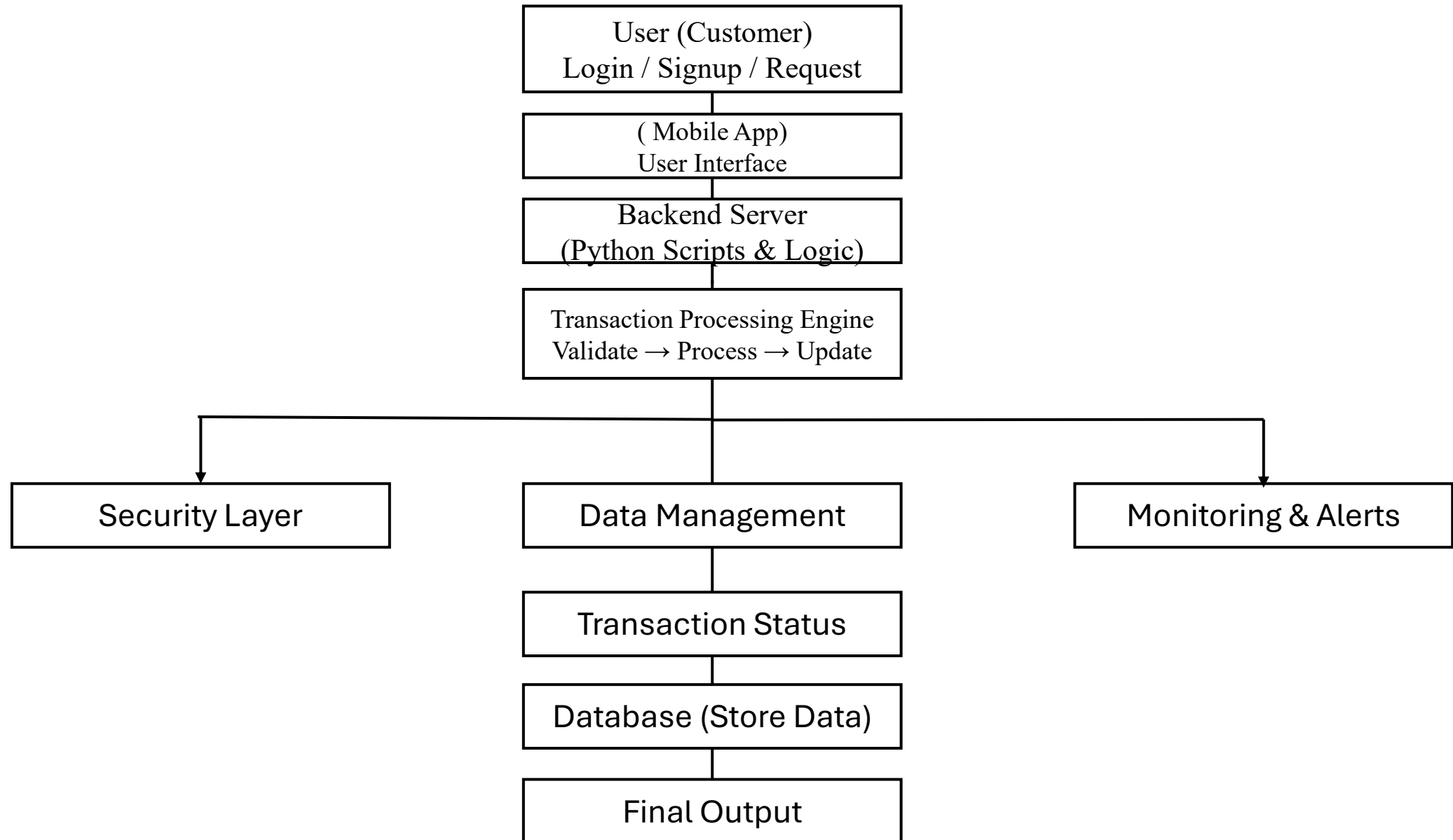
PROBLEM STATEMENT

Traditional banking systems are time-consuming and not easily accessible . Existing digital platforms may have security and data privacy issues . Difficulty in managing user data and transaction records efficiently .Need for a secure, reliable, and user-friendly banking system using Python

LITERATURE SURVEY

S.NO	AUTHOR	PAPER TITLE	METHOD	REMARK
1	M Kamber	Secure Banking Transaction and Digital Banking Using Blockchain Technology(2022)	Blockchain, framework	Improves security
2	GS Kalra	<u>Real-Time Payment Processing Architecture: Building for the Future of Digital Banking(2025)</u>	Analytical and descriptive research	Improve accuracy
3	N Aziz	Encrypting of Digital Banking Transaction Records: An Blockchain Cryptography Security Approach(2021)	Cryptographic Implementation Method	using Python scripts
4	B. Y. Baha	Enhanced Security Framework for Internet Banking Services (2021)	Data Collection Methods	Robust

SYSTEM ARCHITECTURE



MODULE 1:User Authentication Module

- User enters personal and account details through the GUI
- System validates inputs and stores data securely in the MySQL database
- Secure login is performed using username and password authentication
- Customer profile details can be updated when required
- Account status is checked to allow or restrict system access

← → ↻ ⓘ 127.0.0.1:5000

Transaction Stored Successfully

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\jasmi\Downloads\trust_cart> ^C
PS C:\Users\jasmi\Downloads\trust_cart> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 311-101-201
127.0.0.1 - - [25/May/2026 21:55:58] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [25/May/2026 21:56:49] "GET /transactions HTTP/1.1" 200 -
127.0.0.1 - - [25/May/2026 21:57:14] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [25/May/2026 21:57:19] "GET /transactions HTTP/1.1" 200 -
```

```
from flask import Flask, render_template, request
import sqlite3

app = Flask(__name__)

# ----- DATABASE -----

def init_db():

    conn = sqlite3.connect("bank.db")
    cursor = conn.cursor()

    # Accounts table
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS accounts (
        acc_no INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT,
        username TEXT,
        password TEXT,
        balance INTEGER,
        status TEXT
    )
    """)

    # Transactions table
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS transactions (
        tid INTEGER PRIMARY KEY AUTOINCREMENT,
        acc_no INTEGER,
        type TEXT,
        amount INTEGER
    )
    """)

    conn.commit()
    conn.close()

init_db()

# ----- SAMPLE ACCOUNT -----

def insert_sample_data():

    conn = sqlite3.connect("bank.db")
    cursor = conn.cursor()

    cursor.execute("""
    INSERT INTO accounts
    (name, username, password, balance, status)
    VALUES
    ('Jasmitha', 'jasmitha', '1234', 50000, 'Active')
    """)

    conn.commit()
    conn.close()

# Run only once
try:
    insert_sample_data()
except:
    pass
```

MODULE 2: Transaction Processing Module

- Authenticated user selects the required transaction type through the GUI
- System fetches current account balance from the database
- Balance validation is performed before withdrawal or fund transfer
- Transaction is processed securely and account balance is updated
- Transaction details are stored and displayed as transaction history

```
# ----- HOME PAGE -----

@app.route("/", methods=["GET", "POST"])
def index():
    message = ""

    if request.method == "POST":
        username = request.form["username"]
        amount = int(request.form["amount"])

        conn = sqlite3.connect("bank.db")
        cursor = conn.cursor()

        # Find account
        cursor.execute(
            "SELECT acc_no FROM accounts WHERE username=?",
            (username,)
        )

        user = cursor.fetchone()

        if user:
            acc_no = user[0]

            # Insert transaction
            cursor.execute("""
INSERT INTO transactions
(acc_no, type, amount)
VALUES (?, ?, ?)
""", (acc_no, "Deposit", amount))

            # Update balance
            cursor.execute("""
UPDATE accounts
SET balance = balance + ?
WHERE acc_no = ?
""", (amount, acc_no))

            conn.commit()

            message = "Transaction Successful"

        else:
            message = "User Not Found"

        conn.close()

    return render_template("index.html", message=message)

# ----- ACCOUNTS PAGE -----

@app.route("/accounts")
def accounts():
    conn = sqlite3.connect("bank.db")
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM accounts")

    data = cursor.fetchall()

    conn.close()

    return str(data)
```

```
# ----- TRANSACTIONS PAGE -----

@app.route("/transactions")
def transactions():
    conn = sqlite3.connect("bank.db")
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM transactions")

    data = cursor.fetchall()

    conn.close()

    return str(data)

# ----- RUN APP -----

if __name__ == "__main__":
    app.run(debug=True)
```

Transaction History

ID	Username	Amount
1	jasmitha	12000
2	jasmitha	12000
3	jas	32000
4	jasmitha	12000
5	jasmitha	12000
6	jas	1200
7	jasmitha	12000
8	jasmitha	12000
9	jasmitha	5000
10	jasmitha	5000
11	arjun	7000000
12	arun	500000

Conclusion

The Secure Digital Banking and Transaction Processing Platform ensures safe and reliable online banking services through authentication, encryption, and secure transaction processing. It protects user data, improves transaction accuracy, and reduces security risks. The project provides an efficient and user-friendly banking system with scope for future enhancements like AI-based fraud detection and blockchain integration.

Future Scope

- Integration of AI-based fraud detection systems
- Implementation of blockchain technology for secure transactions
- Development of mobile banking applications
- Addition of biometric authentication methods
- Cloud-based storage for better scalability and performance

REFERENCES

- Stallings, W. (2022). *Cryptography and Network Security: Principles and Practice*. Pearson Education.
- Kumar, R., & Singh, P. (2023). “Secure Digital Banking Using Encryption Techniques.” *IEEE Access Journal*.
- Alshamrani, A. (2024). “Cyber Security in Online Banking Systems.” *ScienceDirect Journal of Information Security*.
- Elmasri, R., & Navathe, S. (2021). *Fundamentals of Database Systems*. Pearson Publications.
- Sharma, V., & Gupta, N. (2023). “Transaction Processing and Fraud Detection in Digital Banking.” *International Journal of Computer Applications (IJCA)*.

THANK YOU